

86. Partition List

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: July 10, 2026

Problem Summary

Given the head of a linked list and a value x , partition the list so all nodes with values **less than x** come before nodes **greater than or equal to x** , preserving original relative order within each partition.

Approach

Build two separate lists simultaneously — less collects nodes $< x$, greater collects nodes $\geq x$. Traverse the original list once, append each node to the appropriate list, then join less $\rightarrow$ greater at the end.

Algorithm

1. Create two dummy heads: lessHead and greaterHead.
2. Set less = lessHead, greater = greaterHead, current = head.
3. While current != null:
 - If current.val $< x \rightarrow$ append to less list, advance less.
 - Else $\rightarrow$ append to greater list, advance greater.
 - Advance current.
4. greater.next = null — terminate the greater list.
5. less.next = greaterHead.next — join the two lists.
6. Return lessHead.next.

Key Insights

- Two dummy heads eliminate null checks when appending the first node to each list.
- greater.next = null is essential — the last node in greater still has its old next pointer which could create a cycle.
- Return lessHead.next and greaterHead.next — never the dummies themselves.
- Relative order is preserved naturally since you traverse left to right and always append to the tail.

Observations

- greater.next = null must come **before** joining — otherwise the last greater node may still point somewhere in the original list.
- This pattern (two dummy lists $\rightarrow$ join) is cleaner than trying to relink nodes in-place.
- No new nodes are created — existing nodes are relinked into the two lists.

Points to Remember

1. **Two-list partition pattern:** build less and greater lists with dummy heads, traverse once, join at end — reusable for any "split by condition then merge" linked list problem.
2. Always greater.next = null before joining — forgetting this causes cycles or wrong output.
3. Always return dummyHead.next not the dummy itself.
4. **Dummy head pattern** reused here again — anytime you're building a new linked list from scratch, start with a dummy.
5. Preserve order for free by always appending to tail, never prepending.

Time Complexity & Space Complexity

$O(n)$ — single pass through all nodes.

$O(1)$ — only pointers, no extra data structures.

Linked List

DescriptionEditorialSolutionsSubmissions

86. Partition List

Solved

Medium

Topics

Companies

Given the head of a linked list and a value x, partition it such that all nodes less than x come before nodes greater than or equal to x.

You should preserve the original relative order of the nodes in each of the two partitions.

Example 1:

Input:

head = [1,4,3,2,5,2], x = 3

Output:

[1,2,2,4,3,5]

Example 2:

Input:

head = [2,1], x = 2

Output:

[1,2]

Constraints:

- The number of nodes in the list is in the range [0, 200].
- 100 <= Node.val <= 100
- 200 <= x <= 200

8.1K

189

35 Online

Accepted

All Submissions

Accepted

168 / 168 testcases passed

Sahil Khan submitted at Jul 10, 2026 17:14

Runtime

0 ms | Beats 100.00%

Memory

43.92 MB | Beats 31.33%

Code

Java

```
1 /**
2  * Definition for singly-linked list.
3  * public class ListNode {
4  *     int val;
5  *     ListNode next;
6  *     ListNode() {}
7  *     ListNode(int val) { this.val = val; }
8  *     ListNode(int val, ListNode next) { this.val = val; t
9  * }
10 */
11 class Solution {
12     public ListNode partition(ListNode head, int x) {
13         ListNode lessHead = new ListNode(0);
14         ListNode greaterHead = new ListNode(0);
15         ListNode less = lessHead;
16         ListNode greater = greaterHead;
17         ListNode current = head;
18         while(current != null){
19             if(current.val < x){
20                 less.next = current;
21                 less = less.next;
22             } else {
23                 greater.next = current;
24                 greater = greater.next;
25             }
26             current = current.next;
27         }
28         greater.next = null;
29         less.next = greaterHead.next;
30         return lessHead.next;
31     }
32 }
33
34
```

SavedLn 23, Col 39

Testcase

Test Result

AcceptedRuntime: 0 ms

Case 1Case 2

More challenges

1206. Design Skiplist3455. Shortest Matching Substring